Form Introduction

Dynamic Websites

The Websites provide the functionalities that can use to store, update, retrieve, and delete the data in a database.

What is the Form?

A Document that containing black fields, that the user can fill the data or user can select the data.Casually the data will store in the data base

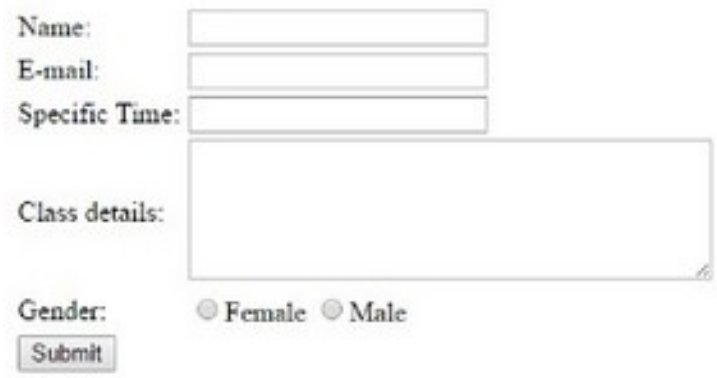
Example

Below example shows the form with some specific actions by using post method.

```
<html>

   <head>
      <title>PHP Form Validation</title>
   </head>

   <body>
      <?php

         // define variables and set to empty values
         $name = $email = $gender = $comment = $website = "";

         if ($_SERVER["REQUEST_METHOD"] == "POST") {
            $name = test_input($_POST["name"]);
            $email = test_input($_POST["email"]);
            $website = test_input($_POST["website"]);
            $comment = test_input($_POST["comment"]);
            $gender = test_input($_POST["gender"]);
         }

         function test_input($data) {
            $data = trim($data);
            $data = stripslashes($data);
            $data = htmlspecialchars($data);
            return $data;
         }
      ?>

      <form method = "post" action = "">
         <table>
            <tr>
               <td>Name:</td>
               <td><input type = "text" name = "name"></td>
            </tr>

            <tr>
               <td>E-mail:</td>
               <td><input type = "text" name = "email"></td>
            </tr>

            <tr>
               <td>Specific Time:</td>
```

```
            <td><input type = "text" name = "website"></td>
         </tr>

         <tr>
            <td>Class details:</td>
            <td><textarea name = "comment" rows = "5" cols = "40"></textarea></td>
         </tr>

         <tr>
            <td>Gender:</td>
            <td>
               <input type = "radio" name = "gender" value = "female">Female
               <input type = "radio" name = "gender" value = "male">Male
            </td>
         </tr>

         <tr>
            <td>
               <input type = "submit" name = "submit" value = "Submit">
            </td>
         </tr>
      </table>
   </form>

   <?php
      echo $name;
      echo "<br>";

      echo $email;
      echo "<br>";

      echo $website;
      echo "<br>";

      echo $comment;
      echo "<br>";

      echo $gender;
   ?>

</body>
</html>
```

It will produce the following result



$_GET

$_GET is one of the superglobals in PHP. It is an associative array of variables passed to the current script via the query string appended to the URL of HTTP request. Note that the array is populated by all requests with a query string in addition to GET requests.

$HTTP_GET_VARS contains the same initial information, but that has now been deprecated.

By default, the client browser sends a request for the URL on the server by using the HTTP GET method. A query string attached to the URL may contain key value pairs concatenated by the "&" symbol. The $_GET associative array stores these key value pairs.

How Does $_GET Work?

When someone visits a URL like this

```
http://localhost/hello.php?name=Amit&age=22
```

PHP can access the values using $_GET.

```
<?php
   echo "Hello, " . $_GET['name'] . "!";
   echo " You are " . $_GET['age'] . " years old.";
?>
```

Output

Here is the outcome of the following code

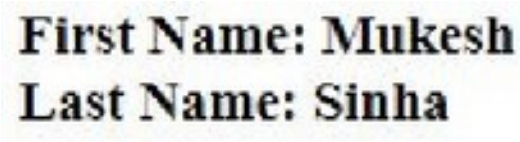
```
Hello, Amit! You are 22 years old.
```

Example

Save the following script in the document folder of Apache server. If you are using XAMPP server on Windows, place the script as "hello.php" in the "c:/xampp/htdocs" folder.

```
<?php
   echo "<h3>First Name: " . $_REQUEST['first_name'] . "<br />" .
   "Last Name: " . $_REQUEST['last_name'] . "</h3>";
?>
```

Start the XAMPP server, and enter "http://localhost/hello.php?first_name=Mukesh&last_name=Sinha" as the URL in a browser window. You should get the following **output**



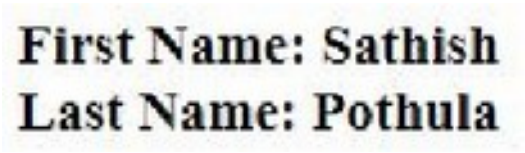
The $_GET array is also populated when a HTML form data is submitted to a URL with GET action.

Under the document root, save the following script as "**hello.html**"

```
<html>
<body>
   <form action="hello.php" method="get">
      <p>First Name: <input type="text" name="first_name"/></p>
      <p>Last Name: <input type="text" name="last_name" /></p>
      <input type="submit" value="Submit" />
   </form>
</body>
</html>
```

In your browser, enter the URL **"http://localhost/hello.html"**

You should get a similar **output** in the browser window



HTML Special Characters

In the following example, htmlspecialchars() is used to convert characters in HTML entities

Character	Replacement
& (ampersand)	&amp;
" (double quote)	&quot;
' (single quote)	' or '
< (less than)	&lt;
> (greater than)	&gt;

Assuming that the URL in the browser is "http://localhost/hello.php?name=Suraj&age=20"

```
<?php
   echo  "Name: " . htmlspecialchars($_GET["name"]) . "";
   echo  "Age: " . htmlspecialchars($_GET["age"]) . "<br/>";
?>
```

It will produce the following **output**

```
Name: Suraj
Age: 20
```

$_POST

The $_POST variable in PHP is a super global array that collects form data after submitting an HTML form using the POST method. It is particularly useful for securely sending data and receiving user input.

What is $_POST?

$_POST is a built-in PHP array. It stores data received from an HTML form using the POST method.This data is not visible in the URL, making it more secure than the GET method.

Syntax

Here is the syntax we can use to define $_POST

```
$_POST['key']
```

Here 'key' is the name of the form input field.

Key Points about $_POST

$_POST is one of the predefined or superglobal variables in PHP. It is an associative array of key-value pairs passed to a URL by the HTTP POST method that uses URLEncoded or multipart/form-data content-type in the request.

- $_POST is a superglobal variable, which means it can be accessed from any point in the script without being marked global.
- Associative array of key-value pairs.
- POST data is not displayed in the URL, making it more secure for sensitive information like as passwords or personal details.
- It is compatible with forms that employ URLEncoded or multipart/form-data content.
- $HTTP_POST_VARS is an old version of $_POST that should not be used.
- The simplest way to transmit data to the server is to modify the HTML form's method attribute to POST.

Example: HTML Form (hello.html)

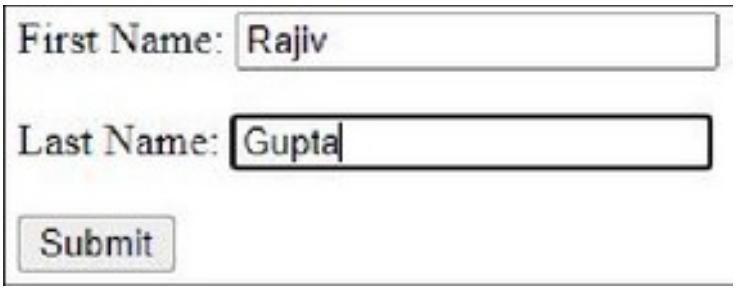
Assuming that the URL in the browser is "http://localhost/hello.php", method=POST is set in a HTML form "hello.html" as below

```
<html>
<body>
   <form action="hello.php" method="post">
      <p>First Name: <input type="text" name="first_name"/> </p>
      <p>Last Name: <input type="text" name="last_name" /> </p>
      <input type="submit" value="Submit" />
   </form>
</body>
</html>
```

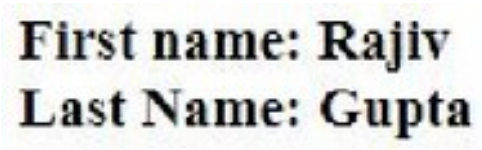
The "hello.php" script (in the document root folder) for this exercise is as follows:

```
<?php
   echo "<h3>First name: " . $_POST['first_name'] . "<br /> " .
   "Last Name: " . $_POST['last_name'] . "</h3>";
?>
```

Now, open **http://localhost/hello.html** in your browser. You should get the following output on the screen



As you press the **Submit** button, the data will be submitted to "hello.php" with the POST method.



You can also mix the HTML form with PHP code in hello.php, and post the form data to itself using the "PHP_SELF" variable

```
<html>
<body>
   <form action="<?php echo $_SERVER['PHP_SELF'];?>" method="post">
      <p>First Name: <input type="text" name="first_name"/> </p> <br />
      <p>Last Name: <input type="text" name="last_name" /></p>
      <input type="submit" value="Submit" />
   </form>
   <?php
      echo "<h3>First Name: " . $_POST['first_name'] . "<br /> " .
      "Last Name: " . $_POST['last_name'] . "</h3>";
   ?>
</body>
</html>
```

It will produce the following **output**